

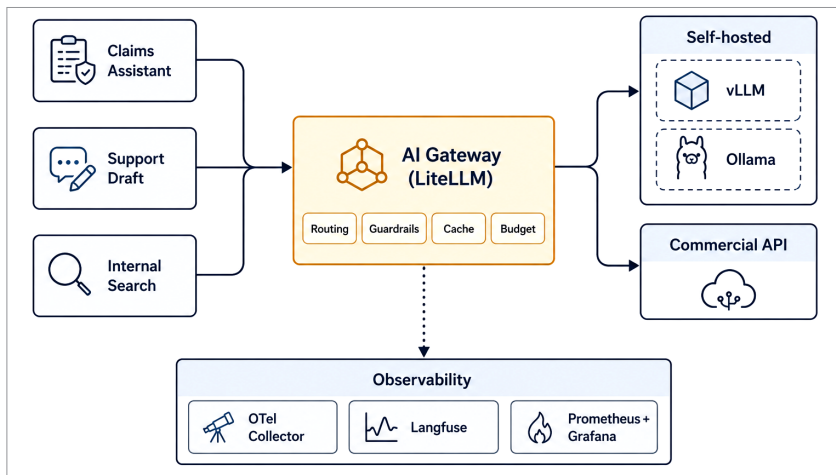


Figure 1: How the gateway sits between applications and models

```

NEXTAUTH_URL: https://llm-obs.
internal.company.in
SALT: ${LANGFUSE_SALT}
TELEMETRY_ENABLED: "false"
  
```

Check that last setting on every tool you install. If you self-host for data residency reasons, it matters. I have seen two air-gapped deployments where three components were still sending usage statistics to a vendor endpoint. Nobody was careless. They had not read that far into the documentation.

On metrics, keep the list short. Track time to first token separately from total latency, because nine seconds to first token feels broken even when the full response arrives at twelve. Token counts as a distribution, not a mean, since a mean hides the one path sending 50,000. Cache hit rate at the gateway warns you before the bill moves. Check tool call depth for agentic workloads, with an alert on it. If you self-host, watch GPU memory and KV cache utilisation rather than CPU.

I gave bad advice about this for a year

Through 2025 I told clients evaluation could wait. Plumbing first: tracing, costs, guardrails, then quality once the base was steady. That was wrong and I have stopped saying it.

What happened was that teams

got everything else right, shipped it, and it all degraded over three or four months. Prompts collect small edits from five or six people, and each edit is defensible alone. By the time someone notices the assistant has gone mediocre, there is no baseline and no way to identify the change that caused it. A team in Bengaluru lost almost a month rebuilding a prompt from memory and old Slack threads.

Run evals in CI: a fixed dataset against every prompt change, merge blocked when the score drops. Sample in production too, because your fixed dataset does not contain the questions users actually ask. Promptfoo is the most CI-friendly option and drops into GitHub Actions like a linter. Ragas is built for retrieval, and gives faithfulness and context relevance scores. DeepEval is pytest-shaped.

```

yaml
- name: Run prompt evaluations
  run: |
    npx promptfoo eval \
      --config evals/support-assistant.
yaml \
  --output results.json
npx promptfoo eval --assert-
threshold 0.85
  
```

Thresholds take a few weeks to tune and the team will be annoyed with you

for a fortnight. Sit through it.

The document tells your model what to do

Direct injection is a user typing something clever into your chat box. Indirect injection is that text sitting inside a document, web page, email or database row that your retrieval step fetched. The second is the real danger: the attacker never touches your application, and your retrieval layer was built by people thinking about relevance rather than trust.

There is no clean fix, whatever a vendor tells you. Keep system instructions and untrusted content in separate message roles rather than concatenating everything into one string. Run a classifier at the gateway, whether Llama Guard, Prompt Guard or something small you tune yourself. Then assume something gets through and limit what it can do.

That last part matters most. Real damage rarely comes from the model saying something rude. It comes from the model calling a tool it should not have called, with parameters it chose. The defence is authorisation, not a better prompt. Tool calls should run with the end user's permissions rather than the service account's, which means threading identity through the whole chain. Reads can be automatic, writes need confirmation, deletes should be hard. Code execution goes in a sandbox: gVisor, Firecracker, or at minimum a stripped container with a hard timeout and no network.

Deny egress by default from inference workloads.

```

yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: agent-egress-restricted
  namespace: llm-apps
spec:
  podSelector:
    matchLabels:
  
```